Introdução

Algoritmos



"Processo de cálculo, ou de resolução de um grupo de problemas semelhantes, em que se estipulam, com generalidade e sem restrições, as regras formais para a obtenção do resultado ou da solução do problema"

Dicionário Aurélio

Algoritmos

algoritmo

al·go·rit·mo

sm

1 MAT , OBSOL Sistema de notação aritmética com algarismos arábicos.

2 MAT Processo de cálculo que, por meio de uma sequência finita de regras, raciocínios e operações, aplicada a um número finito de dados, leva à resolução de grupos análogos de problemas.

3 MAT Operação ou processo de cálculo; sequência de etapas articuladas que produz a solução de um problema; procedimento sequenciado que leva ao cumprimento de uma tarefa.

4 LÓG Conjunto das regras de operação (conjunto de raciocínios) cuja aplicação permite resolver um problema enunciado por meio de um número finito de operações; pode ser traduzido em um programa executado por um computador, detectável nos mecanismos gramaticais de uma língua ou no sistema de procedimentos racionais finito, utilizado em outras ciências, para resolução de problemas semelhantes.

5 INFORM Conjunto de regras e operações e procedimentos, definidos e ordenados usados na solução de um problema, ou de classe de problemas, em um número finito de etapas.

MICHAELIS



"Besides merely being a finite set of rules that gives a sequence of operations for solving a specific type of problem, an algorithm has five importante features:"

1) Finiteness

2) Definiteness

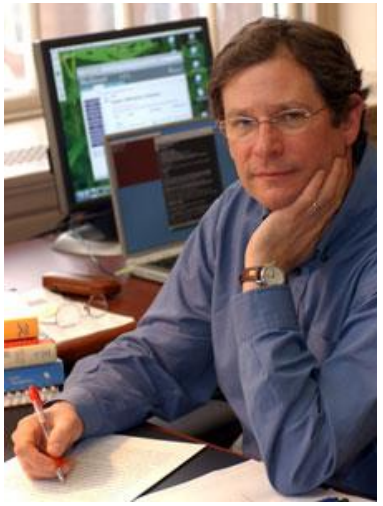
3) Input

4) Output

5) Effectiveness

Algoritmos

"Uma seqüência bem definida de procedimentos computacionais (passos) que levam uma entrada a ser transformada em uma saída"



Cormen *et al*. 2009

Algoritmos

Os passos não podem ser ambíguos.

A saída deve corresponder a uma resposta válida para o problema.

O tempo de execução deve ser finito.

Algoritmos

Algoritmos resolvem problemas



Problemas

O enunciado de um problema especifica em termos gerais o relacionamento entre a entrada e a saída.

Ex: Problema de Ordenação

Entrada: Uma sequência de *n* números $\langle a_1, ..., a_n \rangle$

Saída: Uma permutação (reordenação) da sequência entrada, tal que $a_1 \leq a_2 \leq ... \; a_{n-1} \leq a_n$

Problemas

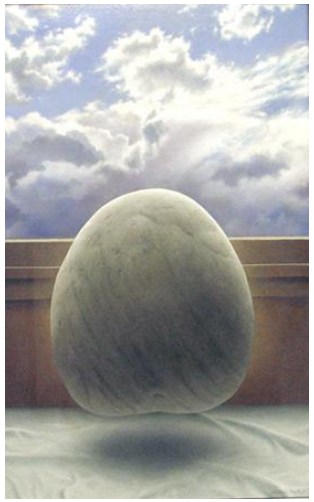
Ex: Casamento de Cadeias

Entrada: Uma sequência X = <x_1, ..., x_n> com n dígitos e uma sequência Y = <y_1, ..., y_m> com m dígitos , $m \leq n$

Saída: Localização da sequência Y em X ou dizer que Y não é subcadeia de X.

Problemas

Ao se fixar os parâmetros de um problema elabora-se um caso concreto ou instância.



Problemas

Multiplicar 4321 por 567 é uma *caso* do problema da multiplicação de dois números inteiros

Ordenar os elementos da lista (12, 34, 86, 77, 4, 25) é uma *caso* do problema de ordenação

Problemas

Outros exemplos de problemas:

1. Solução de um sistema de equações:

O algoritmo deve fornecer uma solução ou dizer que a mesma não existe.

2. Decidir se existe um caminho entre dois pontos em um mapa:

O algoritmo fornece uma resposta: *sim* ou *não*

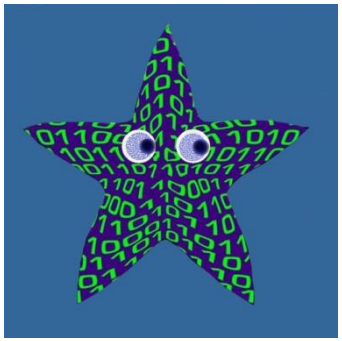
Resolvendo um Problema

1. Compreender o problema
2. Decidir que técnica vai utilizar para desenvolver um algoritmo para o problema (exato, aproximado)
3. Desenvolver o algoritmo
4. Provar que o algoritmo está correto
5. Analisar o algoritmo
6. Codificar o algoritmo

Análise de Algoritmos

Podem existir *vários algoritmos* que solucionem um problema

Analisar um algoritmo significa *predizer* os *recursos* que o algoritmo irá requerer quando executado



Análise de Algoritmos

A preocupação com a “eficiência” de algoritmos supostamente iniciou com os gregos



Juris Hartmanis

Modernamente foi sugerida por *Juris Hartmanis* e *Richard E. Stearns* na década de 60



Richard E. Stearns

Medidas mais comuns: tempo de processamento e quantidade de memória

Análise de Algoritmos

Dado um modelo abstrato de computador (será visto adiante)



Quantificar os recursos computacionais necessários à solução de um problema (todos os casos) como função do tamanho das entradas de suas instâncias

Análise de Algoritmos

Como *recursos* entendemos a *memória*, o *tempo de processamento*, a *natureza das operações*, etc

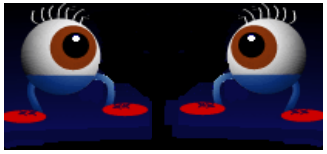


O tamanho de uma instância corresponde ao *número de bits* necessários para *codificá-la* em um computador (*entrada*)

Análise de Algoritmos

A função complexidade deve independer dos recursos de hardware utilizados.

Porque?



Análise de Algoritmos

A análise de complexidade de algoritmos nos auxilia a comparar algoritmos no nível das ideias, ignorando detalhes de baixo nível tais como linguagem de programação, hardware, conjunto de instruções do processador...

Queremos comparar algoritmos em função do que realmente são: ideias de como algo é calculado/computado.

Modelos

Existem diversos modelos de computação como:

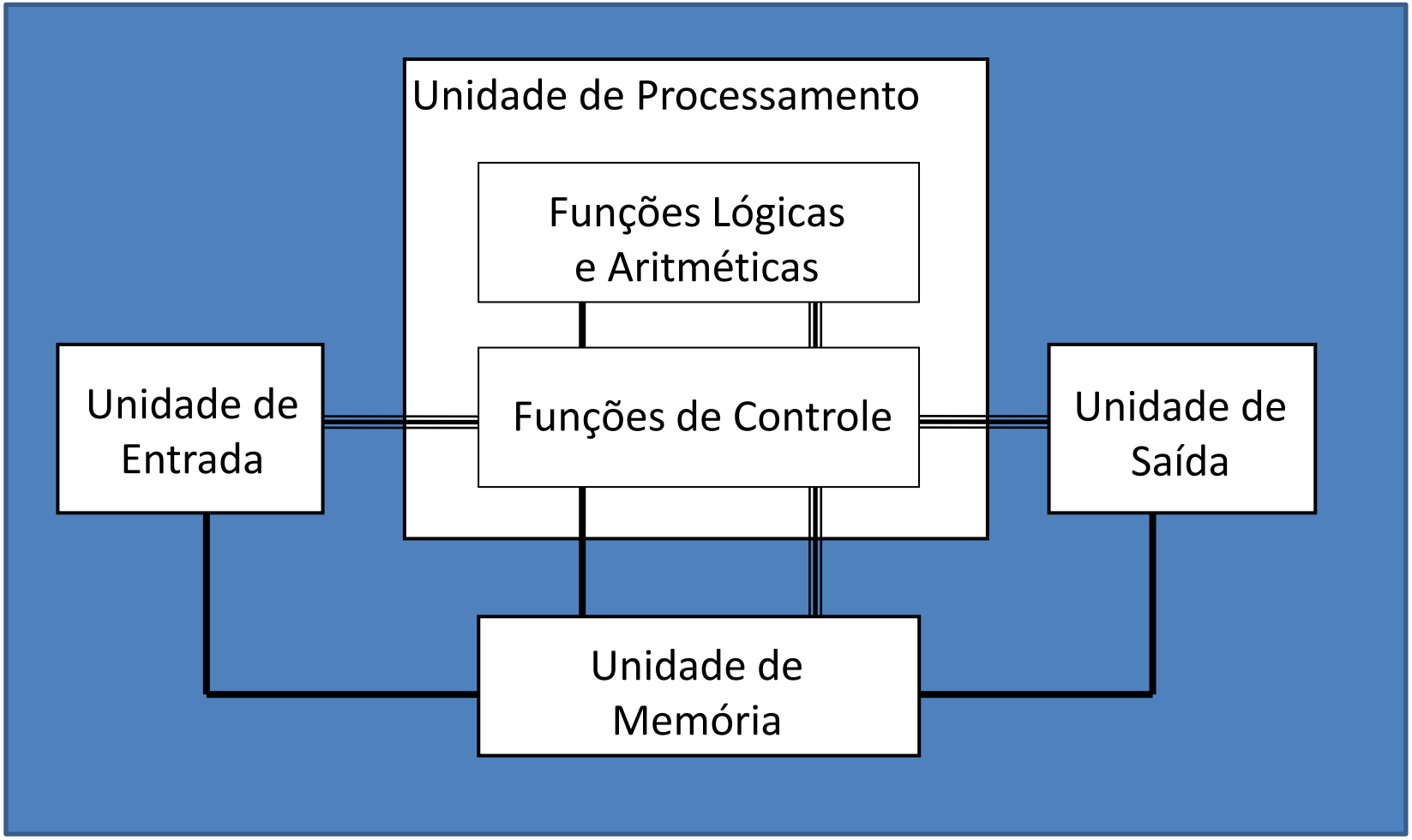
1. Máquina de Turing
2. O modelo RAM (Random Access Machine)
3. A Máquina Baseada em Ponteiros



Alan Mathison Turing

RAM possui um número infinito
de registradores na memória,
cada um podendo guardar um inteiro

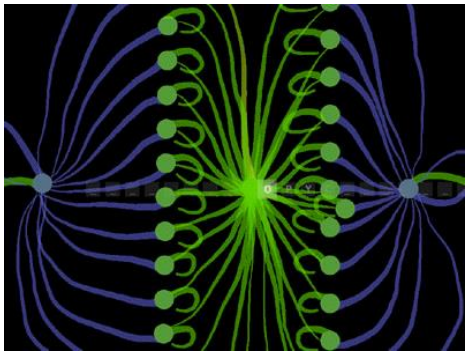
Máquina RAM


Unidade de Processamento
Funções Lógicas e Aritméticas
Funções de Controle
Unidade de Entrada
Unidade de Saída
Unidade de Memória

Máquina RAM

Operações

1. Operações de Entrada e Saída
2. Operações Aritméticas
3. Operações Lógicas e Relacionais
4. Movimentação de Dados entre os Componentes





Máquina RAM

Abstração válida quando:

- O tamanho do problema for suficientemente pequeno para ser armazenado na memória principal do computador
- Os inteiros utilizados nos cálculos forem suficientemente pequenos para serem armazenados em uma única palavra do computador

Complexidade em Tempo

O modelo considera que *cada instrução I* possuirá um *tempo associado t(I)* para ser operacionalizada em RAM

São processadas r_1 instruções do tipo I_1 , r_2 instruções do tipo I_2 , ..., r_m instruções do tipo I_m ,

Então o tempo total de execução *T(n)* será

$$T(n) = \sum_{j=1}^{m} r_j t(I_j)$$

Complexidade em Tempo

t_A: $\{0,1\}^* \rightarrow \aleph$ é dita complexidade do algoritmo A se, para todo x, dado x como entrada de A, A termina em exatamente $t_A(x)$ passos.

Análises:

-Pior Caso

-Melhor Caso

-Caso Médio

Complexidade em Tempo

Pior Caso:

Estamos interessados na dependência da complexidade como função do tamanho da entrada, tomando o valor ***máximo*** de passos para entradas de tamanho relevante.

Sendo n o tamanho da entrada, temos $n \rightarrow \infty$.

Complexidade em Tempo

Melhor Caso:

Estamos interessados na dependência da complexidade como função do tamanho da entrada, tomando o valor ***mínimo*** de passos para entradas de tamanho relevante.

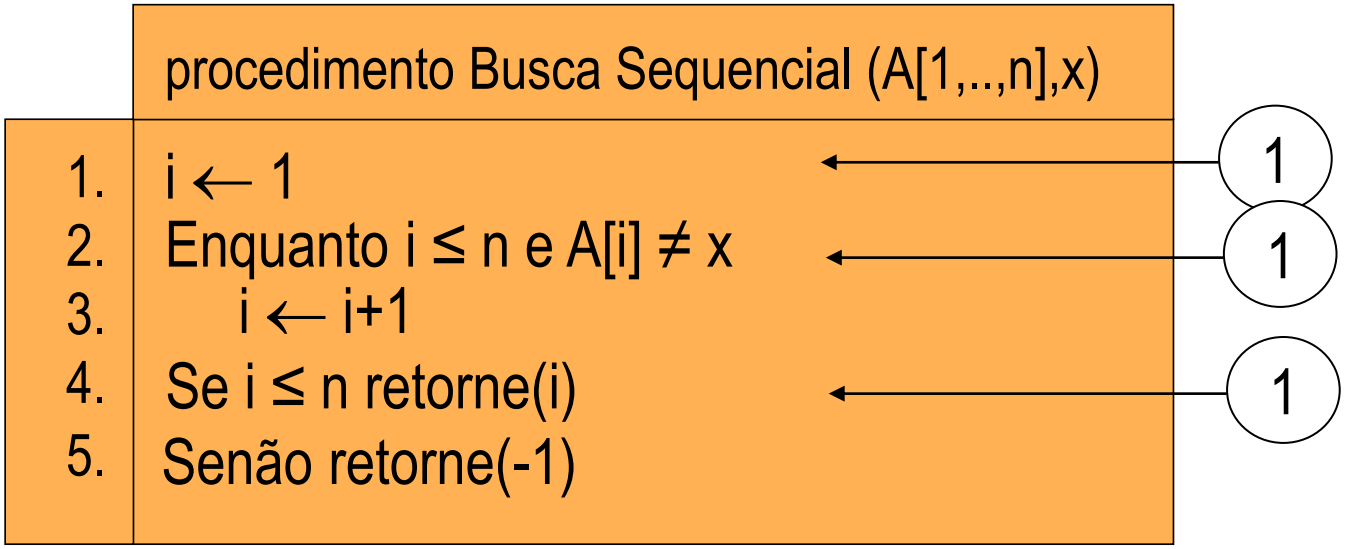
Sendo n o tamanho da entrada, temos $n \rightarrow \infty$.

Exemplo

	procedimento Busca Sequencial (A[1,..,n],x)
1.	$i \leftarrow 1$
2.	Enquanto $i \leq n$ e $A[i] \neq x$
3.	$\quad i \leftarrow i+1$
4.	Se $i \leq n$ retorne(i)
5.	Senão retorne(-1)

Exemplo

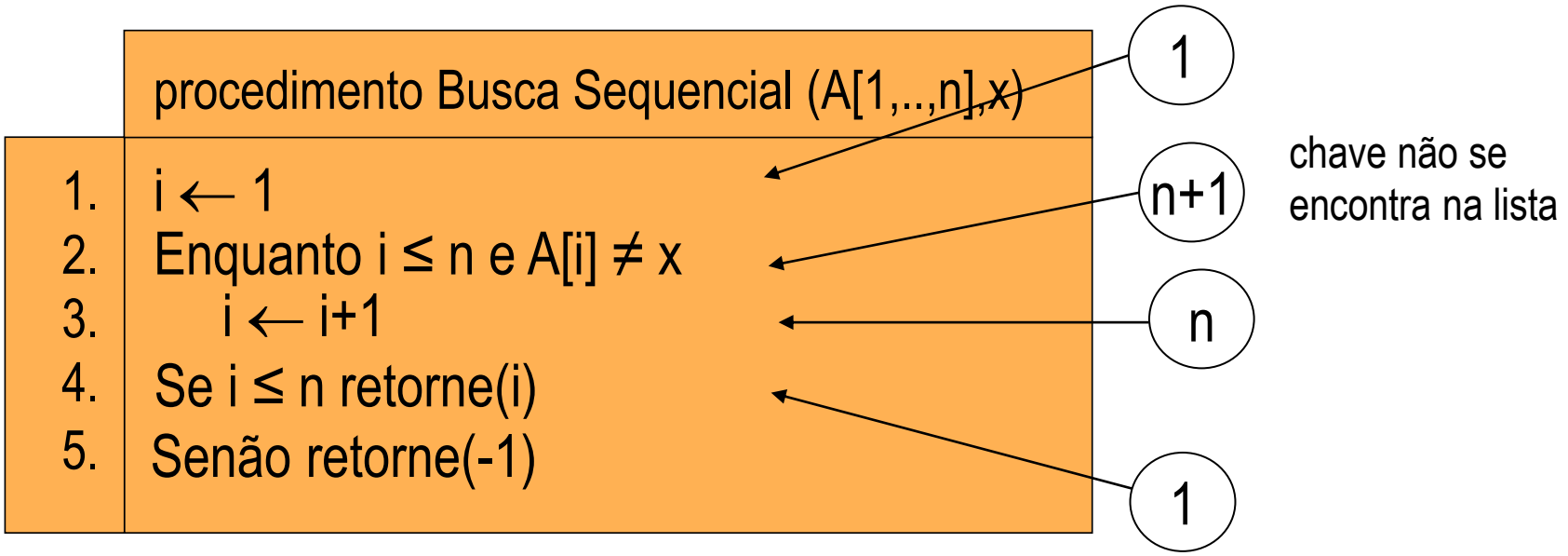
Melhor Caso: A chave de busca se encontra na primeira posição da lista.



$T_{Melhor}(n) = 3$

Exemplo

Pior Caso: A chave de busca se encontra na última posição da lista ou a chave não se encontra na lista.



$T_{Pior}(n) = 2n+3$

Exemplo

Caso Médio: Depende de probabilidades associadas às possíveis entradas para o algoritmo.

Será visto em aula específica.

Exercício

	procedimento Busca SequencialOrdenada(A[1,..,n],x)
1.	i ← 1
2.	Enquanto i ≤ n e A[i] < x
3.	i ← i+1
4.	Se i ≤ n e A[i] = x retorne(i)
5.	Senão retorne(-1)

Melhor Caso?

Pior Caso?

Exercício

```
    procedimento Busca Binária(A[1,..,n],x)
1.  inf ← 1; sup ← n
2.  Enquanto inf ≤ sup
3.      meio ←⌊(inf+sup)/2⌋
4.      Se A[meio] = x retorne(meio)
5.      Se A[meio] < x inf ← meio + 1
6.      Senão sup ← meio - 1
7.  Retorne(-1)
```

Melhor Caso?

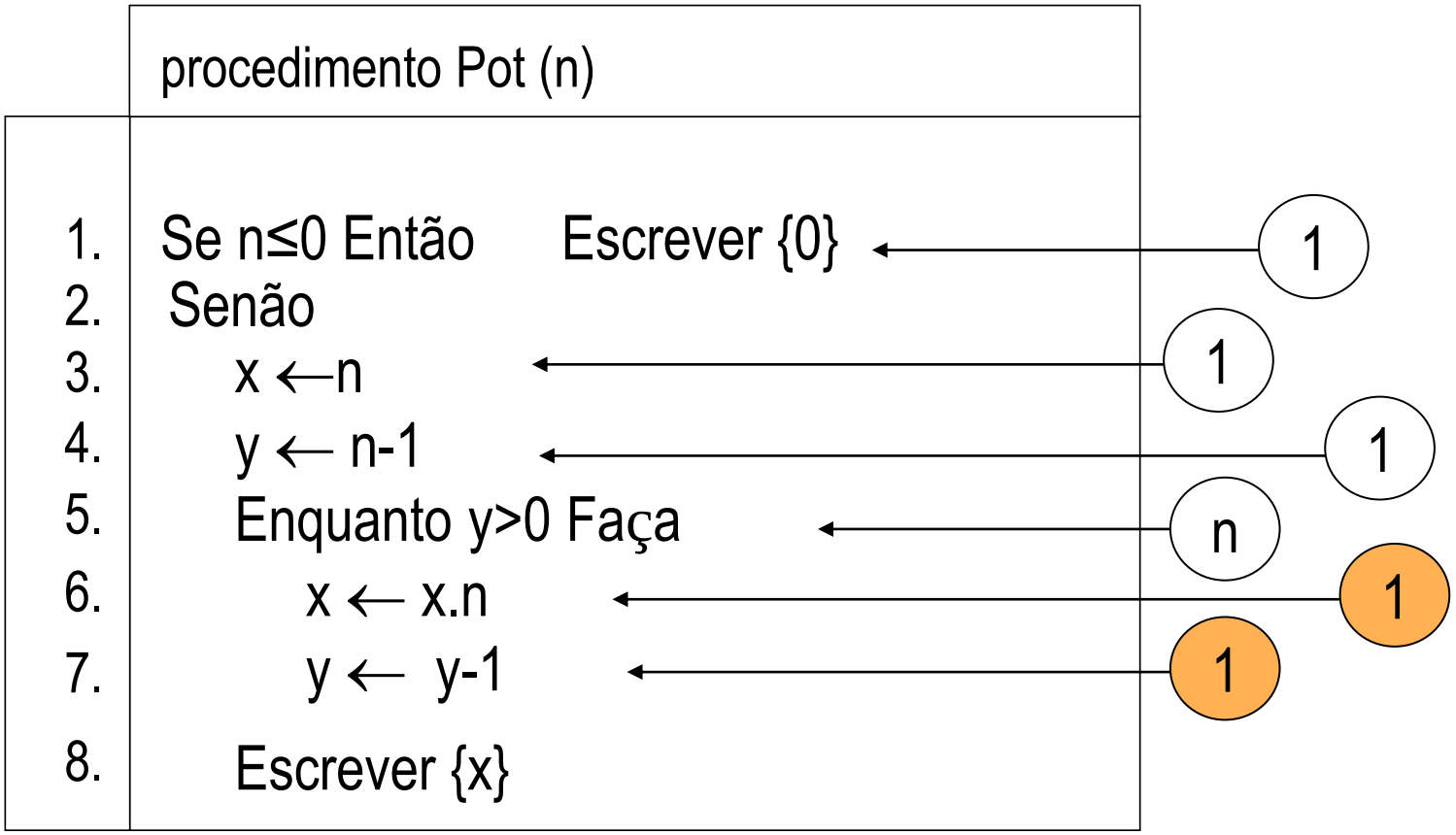
Pior Caso?

Exercício

```
procedimento Pot (n)

1.  Se n≤0 Então      Escrever {0}
2.  Senão
3.      x ←n
4.      y ← n-1
5.      Enquanto y>0 Faça
6.          x ← x.n
7.          y ←  y-1
8.      Escrever {x}
```

Exercício



$T_{Melhor}(n) = T_{Pior}(n) = 3+n+2(n-1) = 3n+1$

Ordenação por Inserção

```
                procedimento inserção (A[1.n])
1.    i = 2;
2.    faça
3.        x = A[i];
4.        j = i - 1 ;
5.        enquanto ((j > 0) e (x < A[j]))
6.
7.            A[j+1] = A[j];
8.             j = j – 1;
9.        faça
10.       A[j+1] = x;
11.       i = i + 1;
12.   enquanto (i <=n);
```

Ordenação por Inserção

```
1.  i = 2;
2.  faça
3.      x = A[i];
4.      j = i - 1 ;
5.      enquanto ((j > 0) e (x < A[j]))
6.
7.          A[j+1] = A[j];
8.          j = j – 1;
9.      fim
10.     A[j+1] = x;
11.     i = i + 1;
12. enquanto (i <=n);
```

procedimento inserção (A[1.*n*])

8	7	1	2	3

i=2

X=7 X=A[2]

j=1

7<A[1]

A[2]=A[1]=8

J=0 ⟶

A[0+1]=7

i=3

7	8	1	2	3

Ordenação por Inserção

7	8	1	2	3

```
    procedimento inserção (A[1.n])
1.  i = 2;
2.  faça
3.      x = A[i];
4.      j = i - 1 ;
5.      enquanto ((j > 0) e (x < A[j]))
6.
7.          A[j+1] = A[j];
8.          j = j – 1;
9.      fim
10.     A[j+1] = x;
11.     i = i + 1;
12. enquanto (i <=n);
```

X=1 i=3

j=2

1<A[2]

A[3]=A[2]=8

J=1

7	8	8	2	3

Ordenação por Inserção

7	8	8	2	3

```
    procedimento inserção (A[1.n])
1.  i = 2;
2.  faça
3.      x = A[i];
4.      j = i - 1 ;
5.      enquanto ((j > 0) e (x < A[j]))
6.
7.          A[j+1] = A[j];
8.          j = j – 1;
9.      fim
10.     A[j+1] = x;
11.     i = i + 1;
12. enquanto (i <=n);
```

j=1>0 x=1<7

A[2]=A[1]=7

J=0 ⟶

A[1]=1

i=4

1	7	8	2	3

Ordenação por Inserção

```
1.  i = 2;
2.  faça
3.      x = A[i];
4.      j = i - 1 ;
5.      enquanto ((j > 0) e (x < A[j]))
6.
7.          A[j+1] = A[j];
8.          j = j – 1;
9.      fim
10.     A[j+1] = x;
11.     i = i + 1;
12. enquanto (i <=n);
```

procedimento inserção (A[1.*n*])

1	7	8	2	3

X=2 i=4

j=3

2<A[3]

A[4]=A[3]=8

J=2

1	7	8	8	3

Ordenação por Inserção

```
1.  i = 2;
2.  faça
3.      x = A[i];
4.      j = i - 1 ;
5.      enquanto ((j > 0) e (x < A[j]))
6.
7.          A[j+1] = A[j];
8.          j = j – 1;
9.      fim
10.     A[j+1] = x;
11.     i = i + 1;
12. enquanto (i <=n);
```

procedimento inserção (A[1.*n*])

1	7	8	8	3

2<A[2]

A[3]=A[2]=7

J=1 2>A[1] ⟶

A[2]=2

i=5

1	2	7	8	3

Ordenação por Inserção

1	2	7	8	3

```
1.  i = 2;
2.  faça
3.      x = A[i];
4.      j = i - 1 ;
5.      enquanto ((j > 0) e (x < A[j]))
6.
7.          A[j+1] = A[j];
8.          j = j – 1;
9.      fim
10.     A[j+1] = x;
11.     i = i + 1;
12. enquanto (i <=n);
```

procedimento inserção (A[1.*n*])

X=3 i=5

j=4

J>0 3<A[4]

A[5]=A[4]=8

J=3

1	2	7	8	8

Ordenação por Inserção

```
1.  i = 2;
2.  faça
3.      x = A[i];
4.      j = i - 1 ;
5.      enquanto ((j > 0) e (x < A[j]))
6.
7.          A[j+1] = A[j];
8.          j = j – 1;
9.      fim
10.     A[j+1] = x;
11.     i = i + 1;
12. enquanto (i <= n);
```

procedimento inserção (A[1.*n*])

1	2	7	8	8

J>0 3<A[3]

A[4]=A[3]=7

J=2 3>A[2] ⟶

A[3]=3

i=6

1	2	3	7	8

Ordenação por Inserção

Prova que o algoritmo é correto

1. Para i = 2 a lista A = [1...i-1] está ordenada.

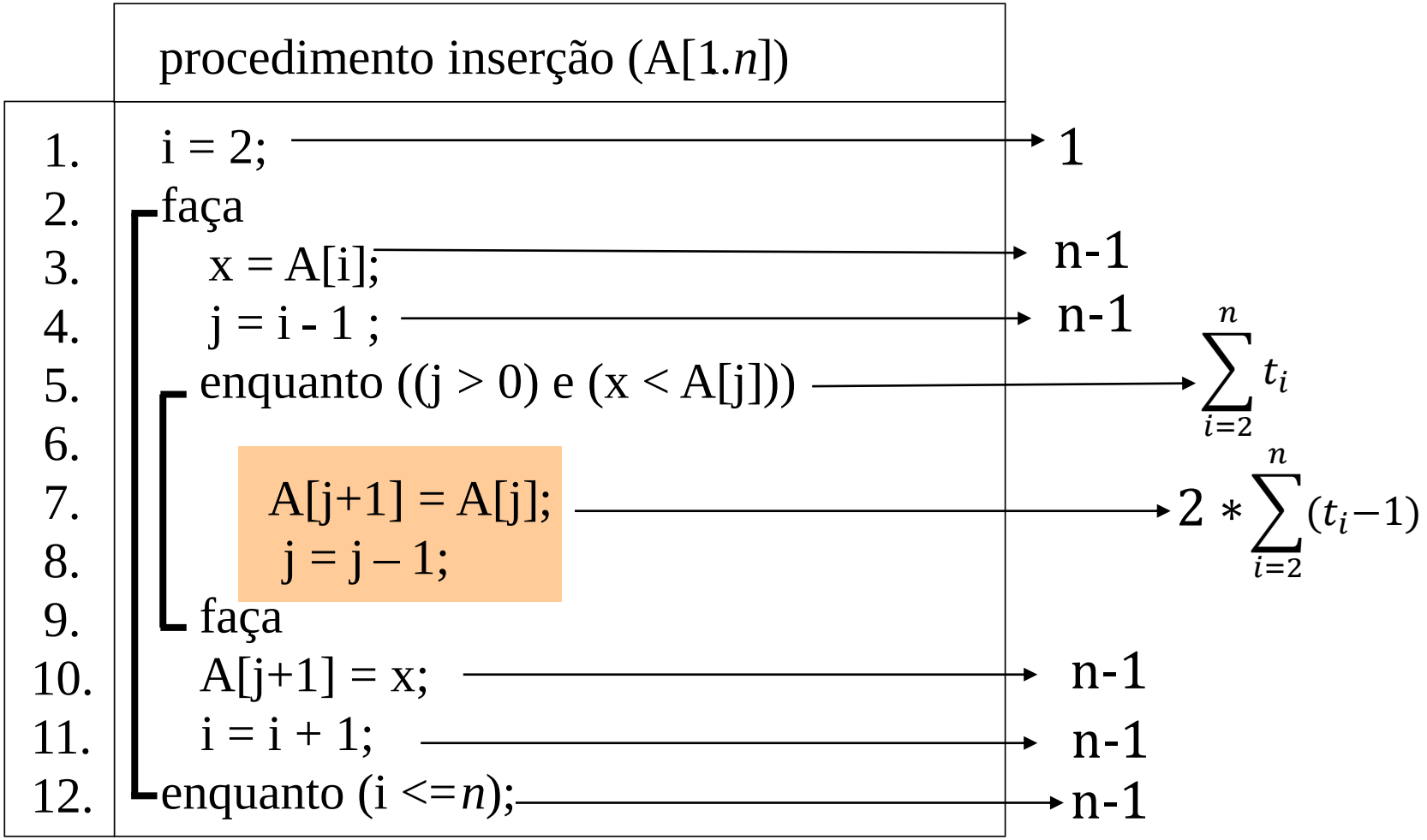
2. O laço externo funciona deslocando os elementos $A[i-1]$, $A[i-2]$,..., uma posição à direita, até encontrar a posição adequada para $A[i]$ onde o elemento é inserido. Portanto, após uma iteração do laço externo, a lista com i elementos está ordenada.

Ordenação por Inserção

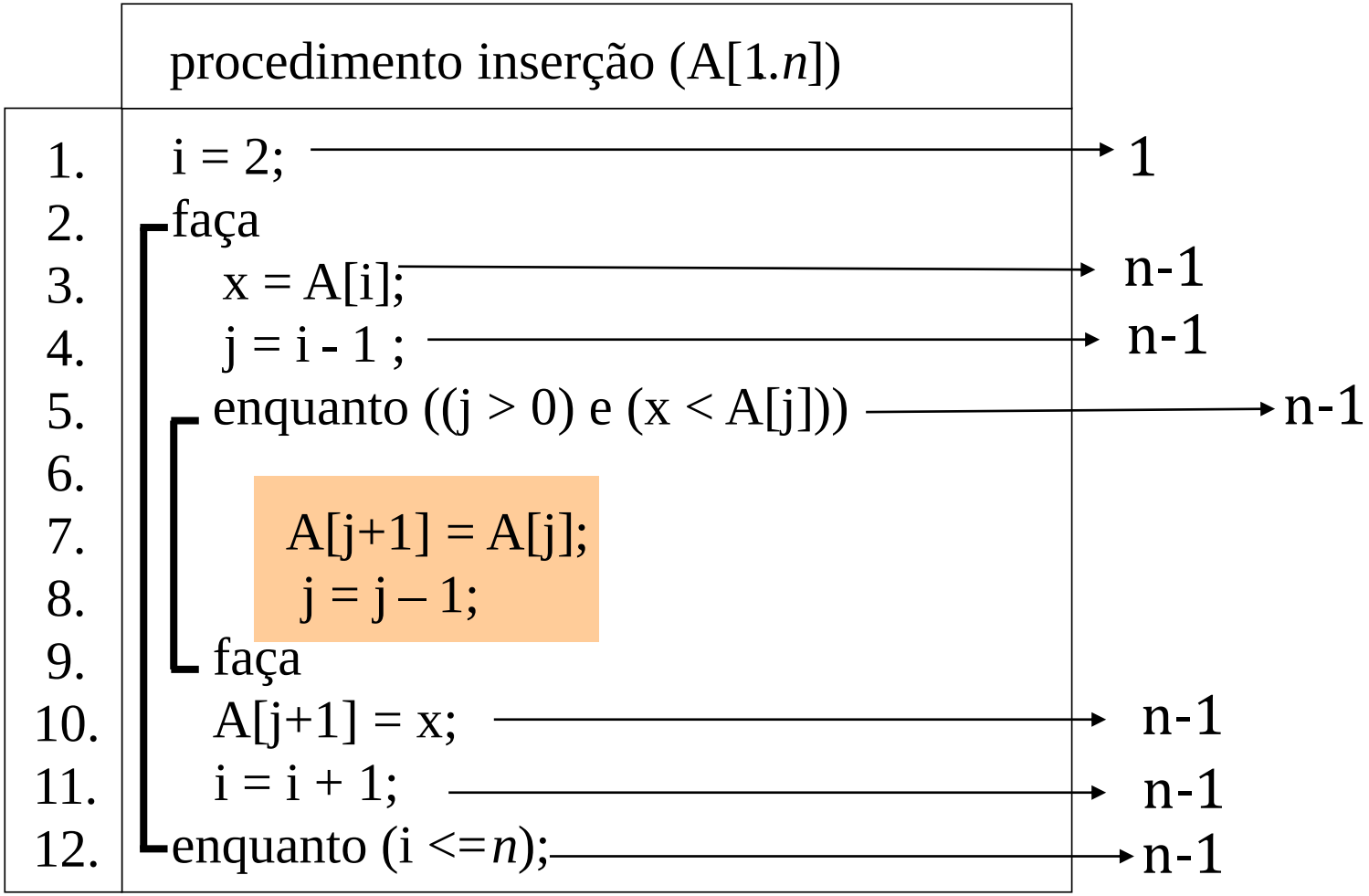
Prova que o algoritmo é correto

3. O laço externo termina quando *i* é maior que o número de elementos em *A*, denotado por *n*. Portanto, a lista com *n* elementos, ao final da iteração do laço externo está ordenada.

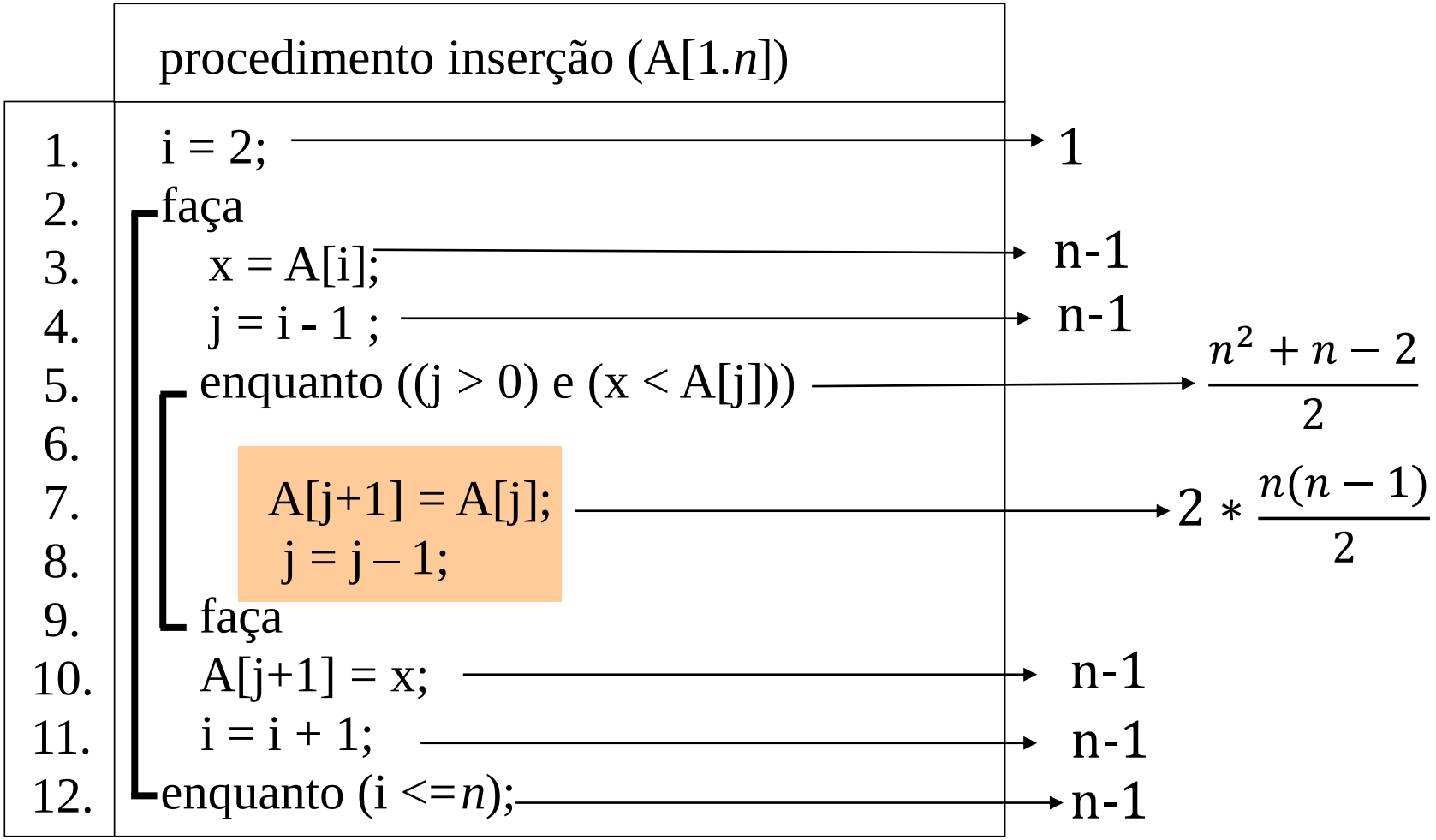
Análise da Ordenação por Inserção



Melhor caso


procedimento inserção (A[1.*n*])
1. i = 2;
2. faça
3. x = A[i];
4. j = i - 1 ;
5. enquanto ((j > 0) e (x < A[j]))
6.
7. A[j+1] = A[j];
8. j = j – 1;
9. faça
10. A[j+1] = x;
11. i = i + 1;
12. enquanto (i <=*n*);
1
n-1
n-1
n-1
n-1
n-1
n-1

Pior caso


procedimento inserção (A[1.n])
1. i = 2;
2. faça
3. x = A[i];
4. j = i - 1 ;
5. enquanto ((j > 0) e (x < A[j]))
6.
7. A[j+1] = A[j];
8. j = j – 1;
9. faça
10. A[j+1] = x;
11. i = i + 1;
12. enquanto (i <=n);
1
n-1
n-1
$\frac{n^2+n-2}{2}$
$2 * \frac{n(n-1)}{2}$
n-1
n-1
n-1

Cálculos

Como os cálculos foram feitos ?

$$\sum_{i=2}^{n} t_i = \sum_{i=2}^{n} i = \frac{(2+n)(n-1)}{2} = \frac{n^2+n-2}{2}$$

$$\sum_{i=2}^{n} (t_i - 1) = \sum_{i=2}^{n} (i-1) = \frac{(1+n-1)(n-1)}{2} = \frac{n(n-1)}{2}$$

O que fazem as funções abaixo?

```
x = A[1];
Para i = 2 até n faça
    se A[i] < x então x = A[i];
```

Função Wilson (*n*)	
Se *n* divide ((*n* -1)! +1)	Exatamente então Retorne “verdadeiro” Senão Retorne “falso”

O que fazem as funções abaixo?

```
x = A[1];
Para i = 2 até n faça
    se A[i] < x então x = A[i];
```

Determina o menor elemento da lista A

```
Função Wilson (n)
Se n divide ( ( n -1 )! +1)  Exatamente então
                             Retorne "verdadeiro"
                             Senão Retorne "falso"
```

Verifica se n é primo

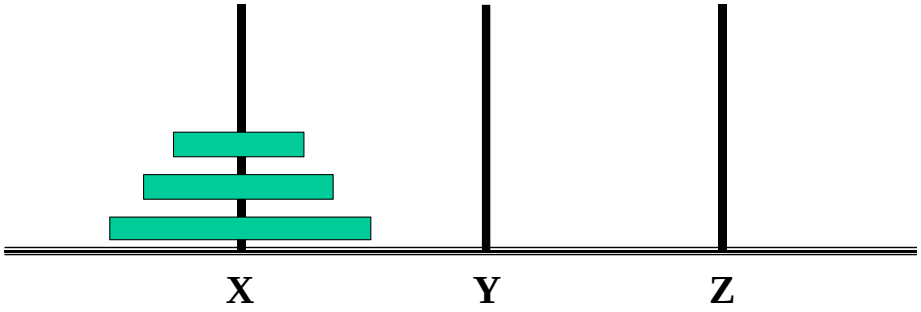
Exercício Proposto

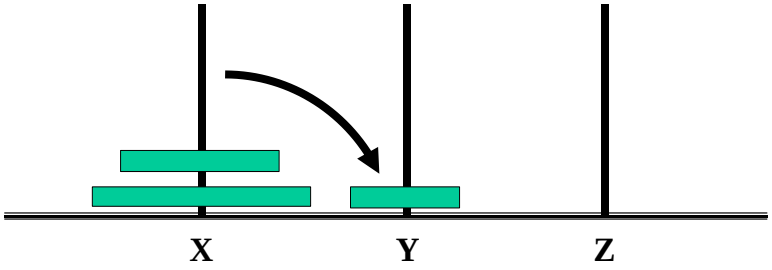
Suponha que estamos comparando implementações de ordenação por inserção e ordenação por intercalação na mesma máquina. Para entradas de tamanho n, a ordenação por inserção é executada em $8n^2$ passos e a por intercalação em $64n\log n$ passos. Para que valores de n a ordenação por inserção supera a ordenação por intercalação?

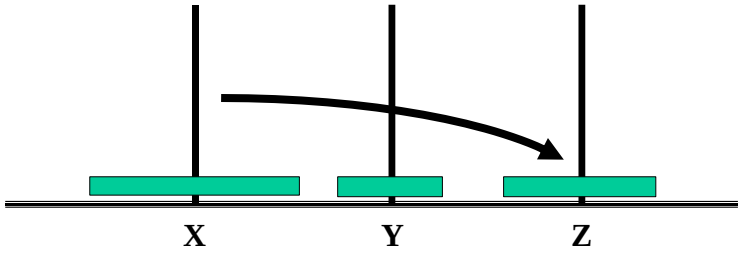
Exercício Proposto

Qual é o menor valor de n para o qual um algoritmo cujo tempo é $100n^2$ executa menos passos que um algoritmo cujo tempo de execução é 2^n na mesma máquina?

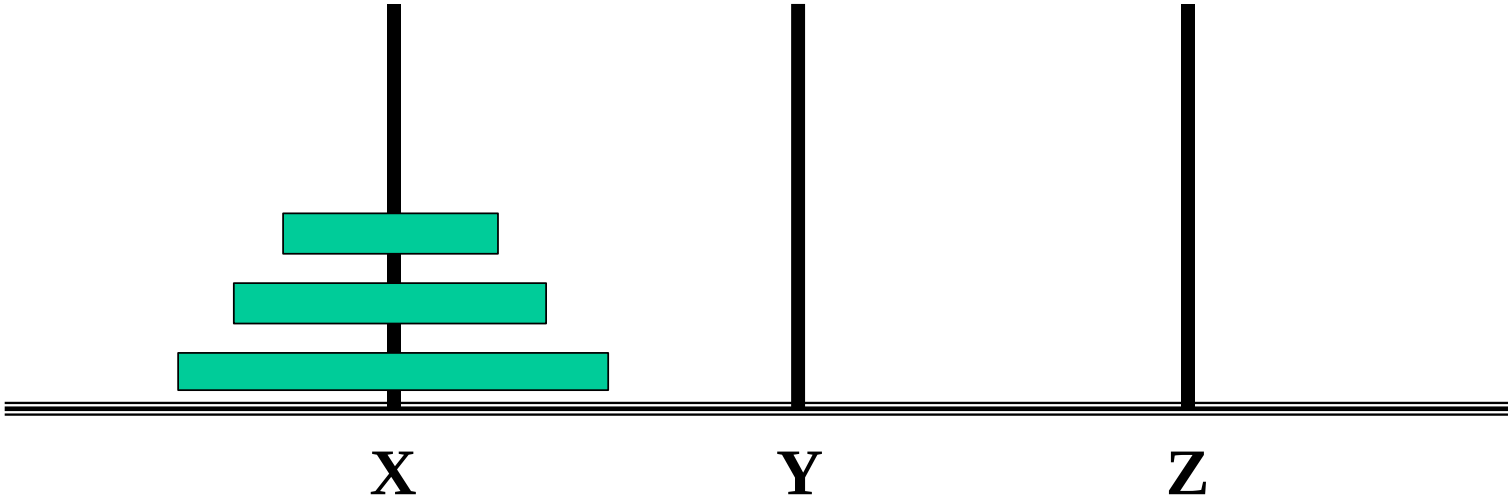
Torres de Hanoi

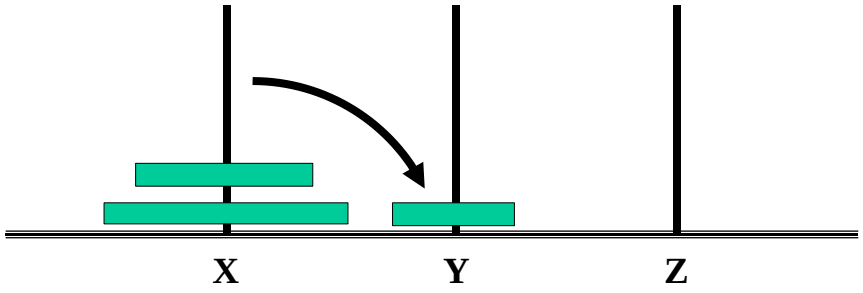


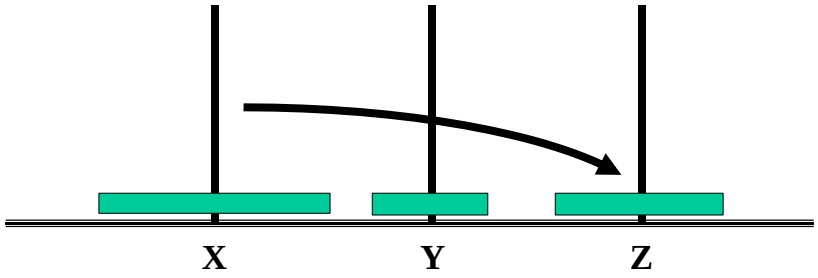




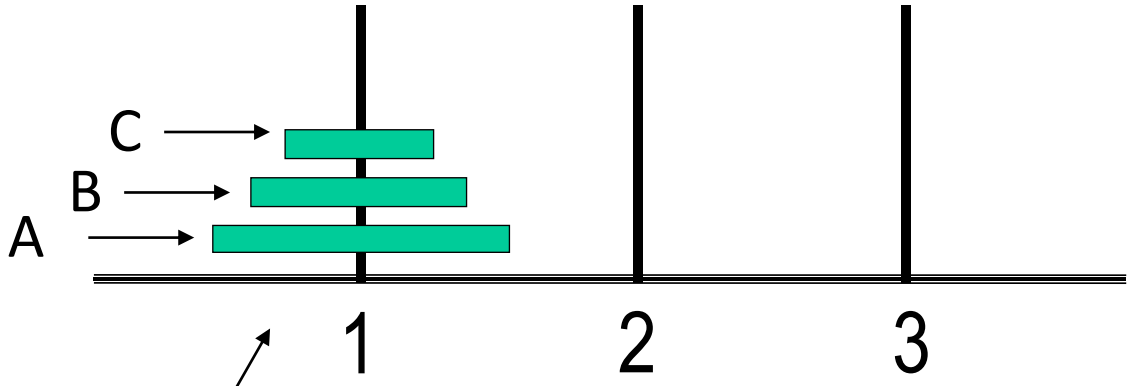
Torres de Hanoi







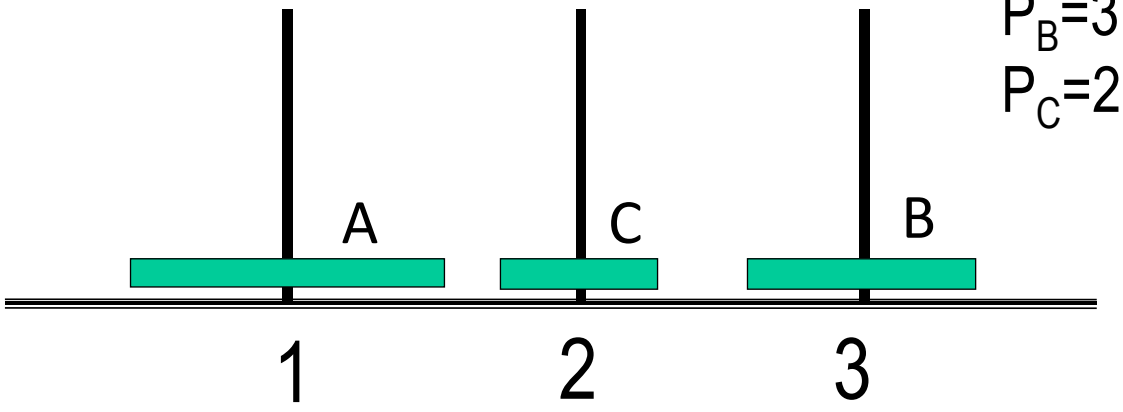
Torres de Hanoi



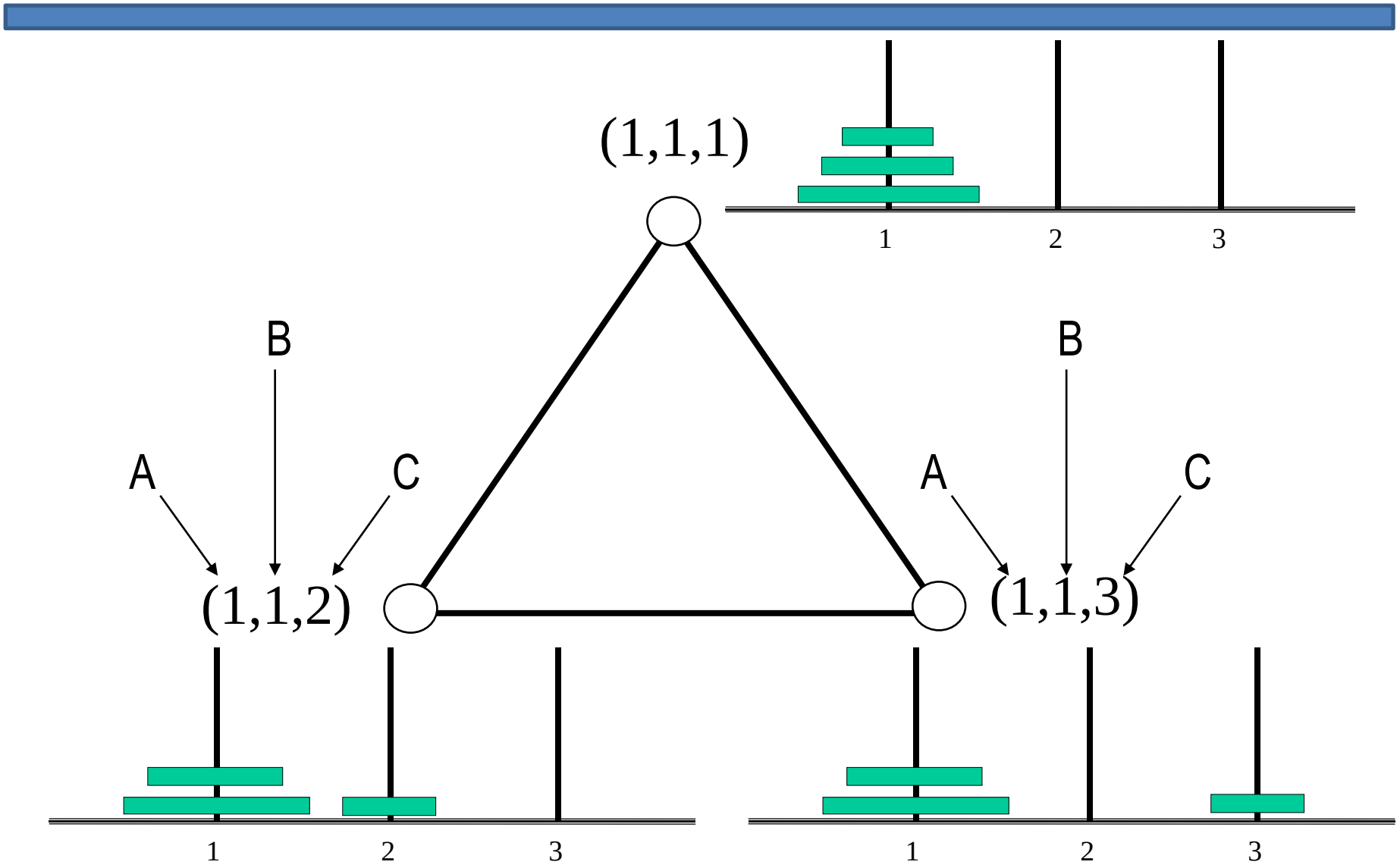
(1,3,2)

(1,1,1)

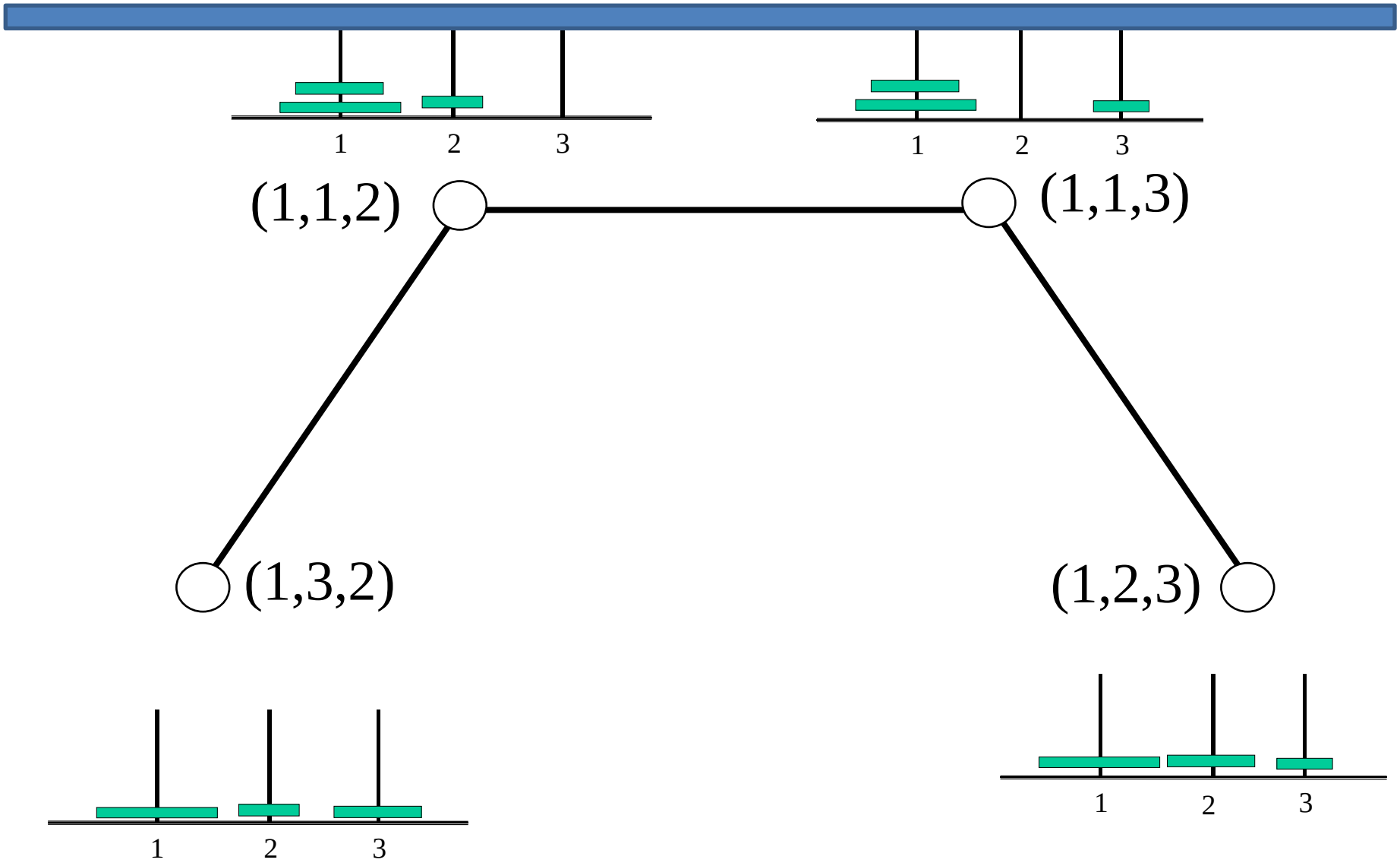
$P_A=1$
$P_B=3$
$P_C=2$



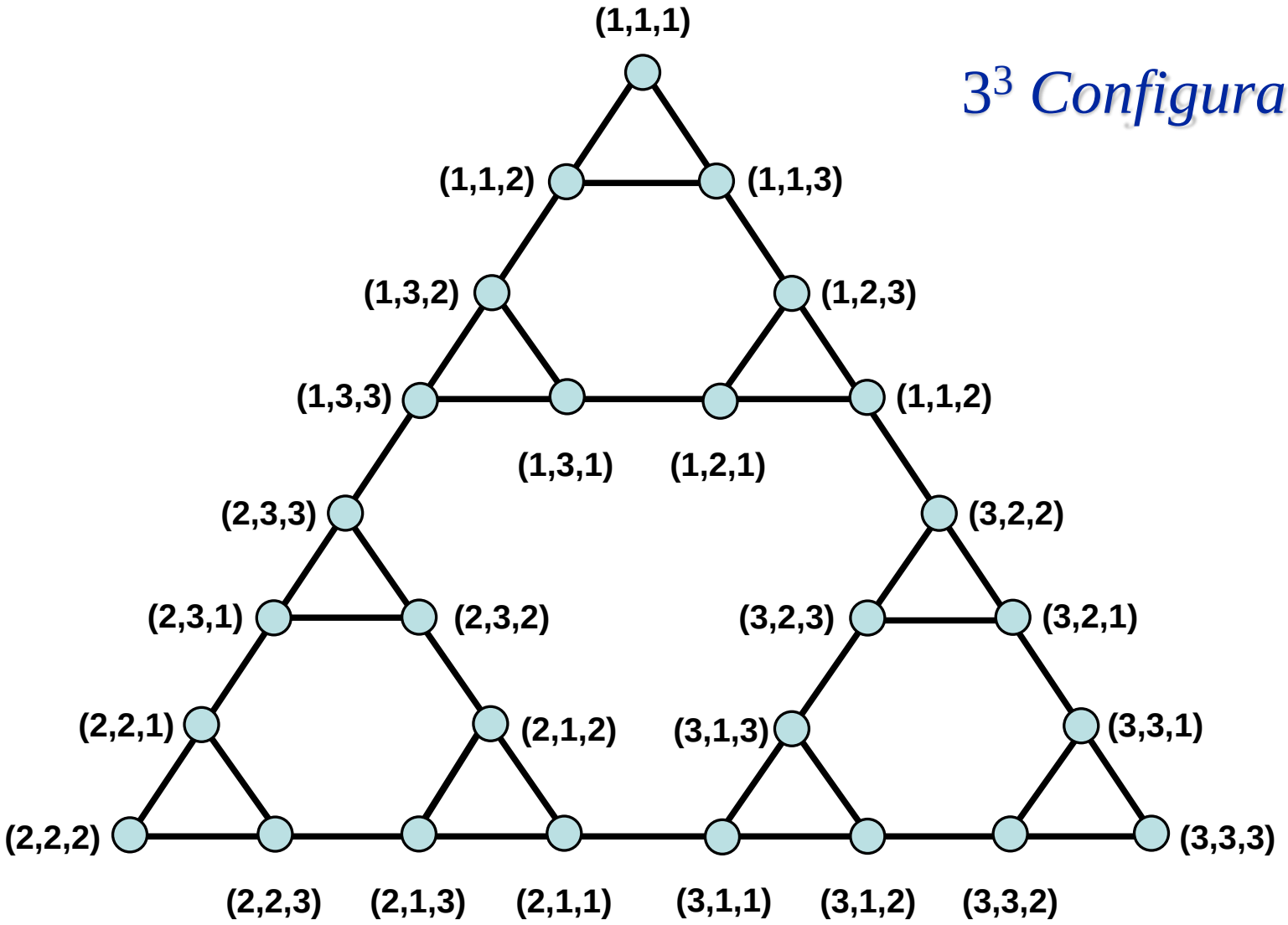
Torres de Hanoi



Torres de Hanoi

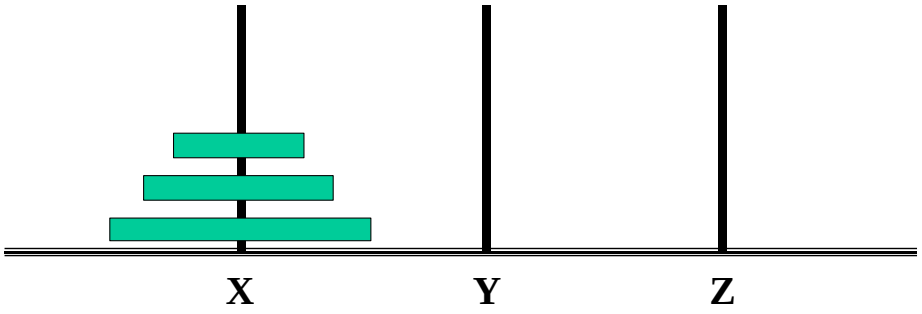


Torres de Hanoi

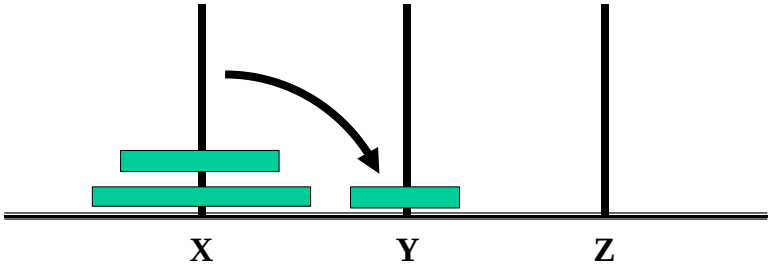


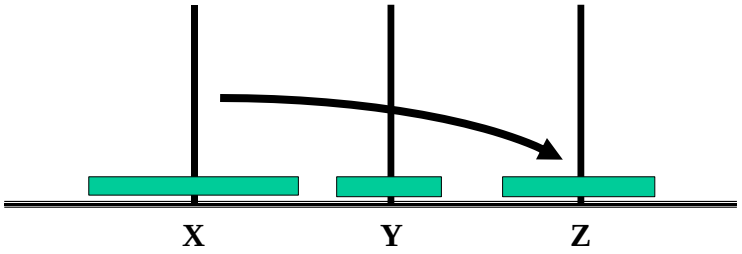
3^3 *Configurações*

Torres de Hanoi

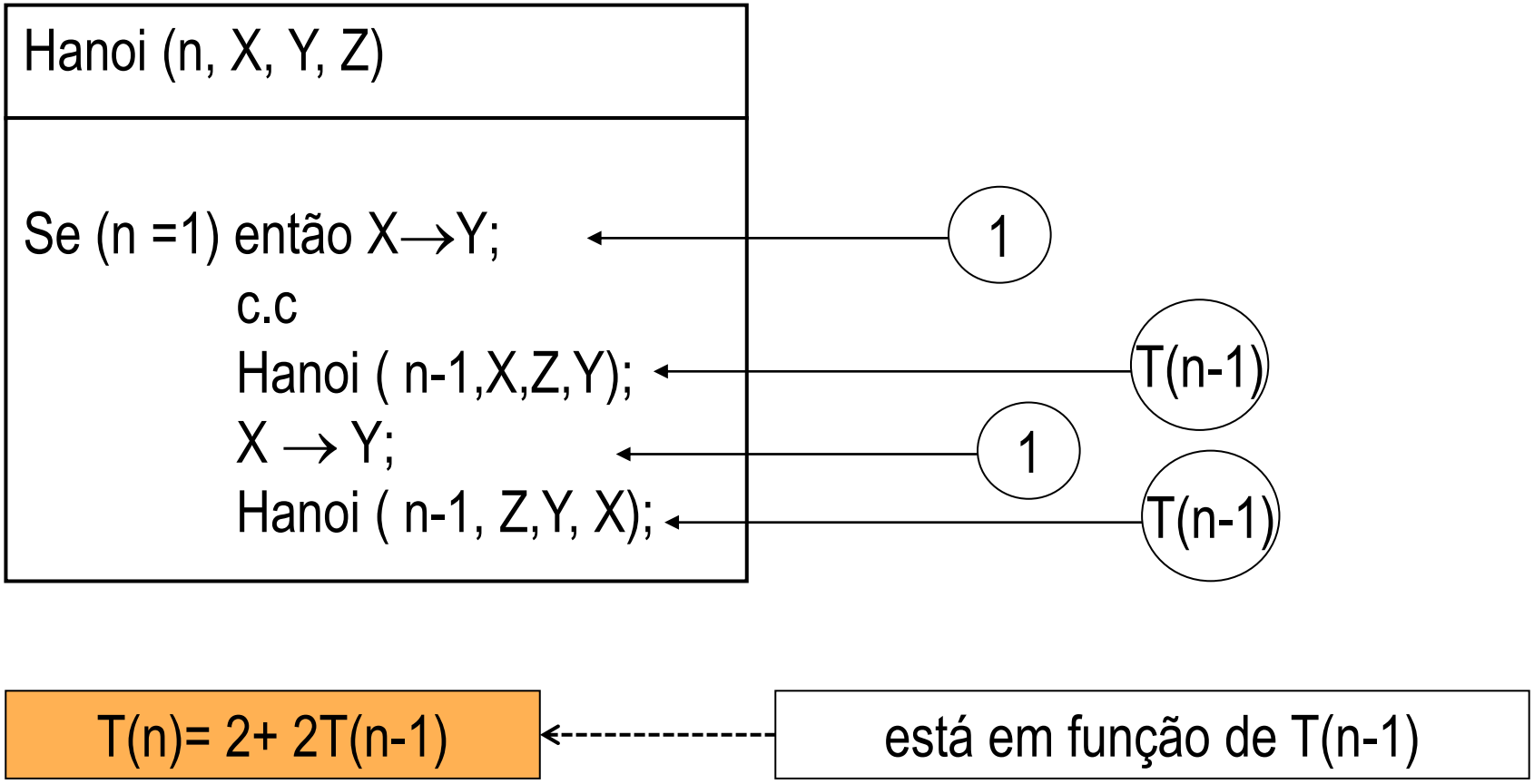


Hanoi (n, X, Y, Z)
Se (n =1) então X→Y; c.c Hanoi (n-1,X,Z,Y); X → Y; Hanoi (n-1, Z,Y, X);





Algoritmo Recursivo



T(n)= 2+ 2T(n-1) ⟵ está em função de T(n-1)

Complexidade em Espaço

Quantidade de memória consumida na computação para guardar resultados intermediários, os quais são armazenados nas células de trabalho (memória).

e_A: $\{0,1\}^* \rightarrow \aleph$ é dita complexidade do algoritmo A se, para todo x, dado x como entrada de A, A acessa exatamente $e_A(x)$ células de trabalho durante a execução de A.